

SuperFedGuard: A Novel Axis of Defense Against Backdoors in Federated Learning

Anonymous Author(s)

Abstract—Super Federated Learning (SuperFL) is a weight-shared approach to federated learning in which clients collaboratively train a SuperNet, to produce higher-accuracy global models at a fraction of the cost of traditional federated learning (TFL). Like TFL, SuperFL is susceptible to data-poisoning attacks in which a fraction of malicious clients steer the aggregated global model toward an adversarial objective such as a backdoor. Defenses against poisoning exist for TFL, but are fundamentally broken in SuperFL. Under weight sharing, each client trains a different subnet each round, so their updates occupy different parameter subspaces. As classical defenses assume all client updates live in the same parameter subspace, their clustering and pairwise-similarity signals are dominated by subnet-topology noise rather than the poisoning signal. However, we find that the same weight-sharing property that disrupts traditional defenses enables a fundamentally new axis of defense that leverages the subnet distribution strategy itself to mitigate poisoning. We present SuperFedGuard, a defense that combines a novel overlap-aware cosine similarity with subnet distribution rules to identify and constrain suspected sybils. Across our experiments, classical defenses modified to operate in the SuperFL setting reduce the attack success rate (ASR) by $\approx 20\%$, whereas SuperFedGuard achieves up to a 65% ASR reduction. We evaluate on CIFAR-10 and MNIST under backdoor attacks across varying malicious-client counts, showing that SuperFedGuard preserves benign accuracy while substantially mitigating the attack.

1. Introduction

Super-Federated Learning [1] (SFL) has been proposed as a faster, more-cost-effective, *accuracy improving* methodology to replace Traditional Federated Learning (FL). Similar to FL [2]–[10], we find that SFL Traditional Federated Learning (FL) systems are alarmingly vulnerable to poisoning attacks, where malicious clients inject manipulated model updates, or poison on-client data, to compromise the central server. In response to these threats, the research community has proposed various defenses which can be broadly categorized as: model analysis methods that vet client updates, robust aggregation rules (AGR) that alter how the global model’s weights are updated, and verification-based methods that introduce new auditing steps for client-submitted gradients and weights [11]–[19]. Such defenses can be categorized along the same axis. They function as robust-AGR-dependent measures that protect against attackers trying to overcome

prior AGRs. This establishes an adversarial arms race along the narrow axis of client-update auditing, where robust-AGRs can be circumvented by attackers aware of the specific defense deployed[DS: add citation here, how do attackers know of the defense beforehand?].

Anomaly detection methods, for example, are designed to filter out updates that are statistically dissimilar from benign ones [12], [14], [17]. This approach may fail when faced with non-IID data, where honest updates are naturally diverse [17], or against stealthy attacks that mimic benign updates [6]. Conversely, defenses based on differential privacy (DP) add noise to updates to obscure malicious contributions; while this can be effective, it often comes at the cost of degrading the model’s accuracy on its primary task [14], [20], [21]. A defense that works well against a targeted-label attack may be completely ineffective against an untargeted attack on the FL system aiming to degrade central server accuracy [3]. This vulnerability to a wide array of attacks stems from a foundational challenge in differentially private FL.

The core principle of FL-DP creates a fundamental blindness for the central server, which cannot audit or verify the training data used in client-side training [20], [21]. This problem is magnified in real-world deployments by data heterogeneity (non-IID). As benign model updates can be naturally divergent and statistically noisy, they provide cover for malicious updates masking as “out-of-distribution” benign updates [10][DS: isn’t this problem stated for anomaly detection methods?]. This creates a trilemma for defenders: any robust defense seems to require either sacrificing model accuracy (e.g., via DP), compromising client privacy (via deeper update inspection), or failing against the next generation of attacks. Consequently, defenders have limited room for moving along the axis of existing AGRs, as all defenses involve auditing/manipulation of client-updates[DS: is this the 2nd point in the trilemma?].

Such defenses are entirely reactive, as after a client has trained its model they either vet the model updates [11], [12] sent to the server or inject noise to reduce the effect of poisoning during aggregation [14]. This defensive posture ignores a critical fact: the central server’s model, in its entirety, is shared with every client including the adversary. Consequently, attacks are not a one-shot effort (accomplished in a single round); they are accomplished through gradual, cumulative changes that are applied to the *exact same set of weights over multiple training rounds*. In traditional FL, every client receives the same full model architecture, which acts as a consistent map for the adversary to accomplish this goal. We make the key insight that, by restricting[DS: obscuring?]

the adversary’s view of the model’s architecture at every round, this mapping is fundamentally broken. Ultimately, this means that adversaries would not be able to push malicious updates targeting the same sets of weights on a consistent round-to-round basis.

We find that because weight-shared SuperNets [1], [22]–[25] provide a natural way of exposing clients to different model architectures at each training round, the effect of malicious-client weight updates on a particular set of weights is reduced. By leveraging SuperNets as a novel paradigm for secure FL where the distribution of subnetworks to clients is an additional lever that defenders can control, more secure FL training can be achieved. Also, by extending existing super-federated learning (SFL) subnetwork distribution strategies[DS: cite] to not only be efficient[DS: is efficiency a goal of this paper?], but also more secure, the natural reduction to the effects of malicious weight updates can be further amplified. Better still, this approach is an orthogonal and complementary defense to existing update auditing and AGR techniques; it acts as a foundational barrier that can be deployed alongside existing defenses to further harden the FL system’s security[DS: this is discussed in the abstract too, so reviewers may ask experiments to back this].

Instantiating this novel approach, we present SuperFedGuard (SFG) which leverages the unique training properties of SFL training to naturally mitigate the effects of central server model poisoning in the FL setting. SFG also implements a novel SFL training methodology to further reduce the effects of malicious client updates. To accomplish this, SFG first applies maximal-overlap subnetwork clustering to identify the subset of weights that are currently shared between all different subnetworks distributed to clients in the current training round (??). Overtime, based on the identified clusters, SFG modifies the distribution of subnetworks to clients; prioritizing non-anomalous[DS: same as benign/non-malicious] client weight updates by increasing their weight-aggregation coefficient and allowing those associated clients to receive larger subnetworks, reducing the size of the set of weights that anomalous[DS: malicious] clients receive over time (??). SFG directly utilizes existing AGR and malicious weight update neutralization strategies[DS: would existing AGRs work well on subnets? because they are designed for the entire server model], but can selectively apply them to subnetworks assigned to potentially anomalous clients, minimizing the impact on subnetworks given to benign clients (??). Importantly, SFG’s methodology is generic to any framework (e.g., future SuperNets) that allows for adaptive distribution of models[DS: subnets?] to clients over time (as discussed in §7).

[DO: Placeholder text from VillainNet just to show size for overall paper length] We evaluate SFG on OFAMobilenetV3 and OFAResnet, SuperNet architectures [22], [23] derived from MobileNetV3 [26] and ResNet [27], using the GTSRB [28] and CIFAR-10 [29] datasets. Furthermore, we evaluate the effects of three different distance metrics between subnetworks during

poisoning to change the attacker-desired effects of the attack. Our experiments demonstrate that SFG successfully achieves attack success rates matching traditional poisoning approaches ($\approx 99\%$) while also achieving granularity of up to 0.004 (lower is better, and naive attack has a granularity of 0.994). We then show that a defender would need to sample on average 66 subnetworks (and upwards of 250 subnetworks for more targeted approaches), to detect the attack, increasing compute costs for detection by $\approx 66\times$. To facilitate future work, and defense strategies, we will make our code, datasets, and model checkpoints open source.

2. Super-Federated Learning Background

2.1. SuperNets and SuperFL

A SuperNet [22], [23], [25] is a single weight-shared neural network that contains, by construction, a family of smaller neural networks (*subnetworks*) with varied dimensions, i.e., depth, width, kernel size, and resolution. A subnetwork can be chosen at inference time by specifying a per-block depth, channel-expansion ratio, kernel size, and input resolution, which together determine which slice of the SuperNets’s parameters is used for the forward pass [22]. Because every subnetwork is a contiguous subset of the SuperNet’s parameters, subnetworks are not parameter-disjoint. A single SuperNet thereby encodes a diverse architectural family (upwards of 10^{19} distinct subnetwork configurations [22]), without requiring any per-architecture retraining.

Training a SuperNet is itself non-trivial because the weight-shared structure causes subnetwork interference: a gradient step that improves one subnetwork can degrade another whose parameters overlap with it [22], [23]. Prior work has proposed progressive shrinking [22] as a solution, which first trains the largest subnetwork end-to-end and then progressively fine-tunes the SuperNet to also support smaller sub-networks while preserving larger subnetwork performance. Subsequent SuperNet-training methods retain the same shared-weight structure but coordinate sub-network sampling and parameter aggregation to further control interference [23].

Super-Federated Learning (SuperFL) is the federated learning paradigm in which the global model is a SuperNet [1]. At each training round, the central server samples a subnetwork configuration for each participating client, materializes the corresponding slice of the SuperNet’s parameters, and sends only that slice to the client. The client trains the assigned subnetwork locally on its own data partition and returns the updated weights to the server. The server then aggregates these heterogeneous-shape updates into the shared SuperNet under a weighted scheme designed to preserve all subnetworks simultaneously. Because the SuperNet’s structure causes subnetworks to interfere with one another during shared-weight training, SuperFL training algorithms

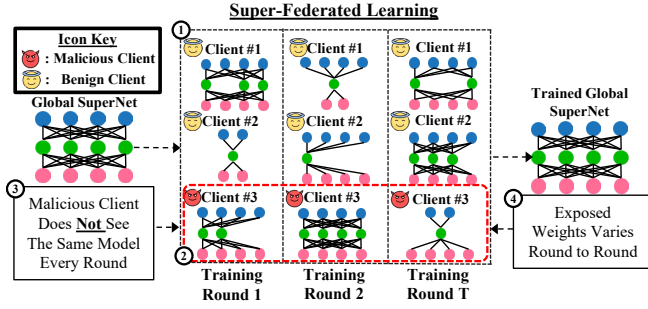


Figure 1: SuperFL provides training-cost and security benefits relative to TFL. Importantly, SuperFL enables client weight-exposure-control as a novel defense lever against backdoor attacks in ??.

such as MaxNet [1] explicitly minimize the loss of the worst-performing subnetwork on each data partition, exploiting the weight-sharing relationship so that gains on smaller subnetworks transfer to the larger ones that contain them.

2.2. Comparing TFL and SuperFL

We highlight the key similarities and differences between traditional FL and SuperFL in Figure 1. ① in Figure 1, shows three clients (out of N total clients) participating in SFL. Traditional FL and SuperFL share the following properties during training: 1) at each training round $t \in T$, each client $n \in N$ contributes weight updates that affect the performance of the central server’s global model, 2) clients $n \in N$ train locally on non-iid data and submit local weight updates to the weight aggregator, 3) the weight aggregator applies some method of weight aggregation (which may adopt prior robust-AGR rules/defenses) and updates the global model accordingly, and 4) after T training rounds (after model convergence) all model weights in the global model are updated (excluding intentionally frozen layers).

2.2.1. SuperFL and TFL Differences. We highlight the key differences between TFL and SuperFL through the lens of a defender. Figure 1 shows an example of an attack on a SuperFL system, where client #3 is a compromised client that aims to have the global model learn an adversary-chosen backdoor. As opposed to in TFL, each client (and therefore the compromised client) does *not* see the same model on a round-to-round basis (③). Notably, this means that in SuperFL there is no guarantee that all weights are updated in the global SuperNet in a single training round. This property of SuperFL is a double edged sword. First, it naturally restricts the exposure of the global SuperNet’s weights to any particular client (④), as the probability of a client receiving the same set of weights in consecutive training rounds is $\approx 0\%$ (assuming the number of subnetworks in the SuperNet mirrors the 10^{19} subnetworks found in a typical SuperNet [22], [23], [25] and the central server applies a randomized subnetwork

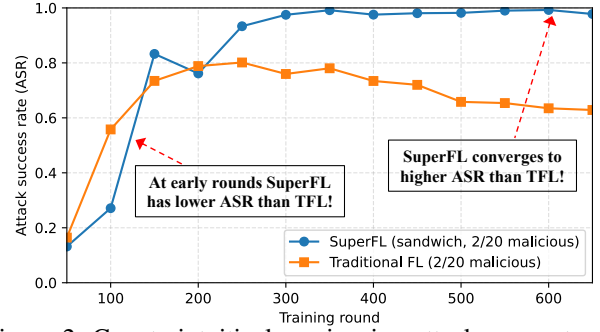


Figure 2: Counterintuitively, poisoning attacks are naturally mitigated more in traditional FL than in SuperFL.

distribution strategy). Second, if the central server *is aware of a potentially malicious client*, the central server can use a non-randomized subnetwork distribution strategy to further restrict the exposure of the SuperNet’s weights to the suspect client (e.g., give suspect clients repeat subnetworks). However, even with these advantages, we find that a SuperFL system (without any modifications) is inherently worse than a TFL system at naturally mitigating poisoning attacks.

2.2.2. A Poisoning Attack Against TFL and SuperFL. We demonstrate through poisoning attacks on both a TFL and SuperFL system that those against the SuperFL system impact the central server’s model more than that of the TFL system (further described in §5.1).

Experimental Setup. We used a once-for-all-style (OFA) ResNet-50 [27] SuperNet with elastic depth $d \in \{0, 1, 2\}$ (per block for up to four blocks) and elastic expansion ratio $e \in \{0.10, 0.14, 0.18, 0.25\}$ for the SuperFL system’s central server’s model. As TFL does not produce a family of models like SuperFL, for the TFL system’s central server’s model we used the ResNet-50 architecture corresponding to the MaxNet configuration in our SuperFL setup (the subnetwork of OFA-Resnet-50 when the depth is two and elastic expansion ratio is 0.25 across all blocks). For both experiments, each FL system was trained with 20 clients on the CIFAR10 [29] dataset until convergence. We chose two clients (10% of the total number of clients) to be malicious. Each malicious client trained their local model on a poisoned CIFAR10 dataset where 10% of images contain a simple trigger [30] and have a corresponding flipped label. In the TFL experiment each client received the same model architecture at every round (matching the architecture of the central server’s model). In the SuperFL experiment each client received a subnetwork of the global SuperNet following the sandwich sampling algorithm [23]. To measure attack success rate (ASR), we evaluate both FL systems on *only poisoned data* at every 50 rounds of training.

Results. Figure 2 highlights the results of training both FL systems over 650 rounds. We see that though SuperFL has lower ASR than TFL in the early training rounds (rounds $[0, 125]$), the ASR in TFL plateaus and then decreases to

$\approx 61\%$ by round 650. However, the SuperFL ASR continues to increase throughout training, converging to $\approx 100\%$ by round 650. These results are counterintuitive, as reduction of the attack surface of the model (e.g., reducing the number of weights a malicious client is able to affect round-to-round) intuitively *should* reduce the post-training ASR of the attack. However, these results highlight the opposite: The poison is not trapped in whatever subnetwork the adversary held in a given round; it accumulates into the larger subnetworks the adversary never directly trained.

Why Rotation Backfires. Two properties of weight sharing turn SuperFL’s defining feature into a liability, both of which the design of §5 later turns back into defenses. First, subnetworks are nested and overlapping rather than disjoint, so a gradient an adversary applies to one round’s subnetwork lands on weights that other subnetworks, including the MaxNet, also read. Over many rounds the adversary’s *cumulative* footprint is the union of every subnetwork it was ever assigned, which approaches the full SuperNet; the per-round attack surface shrinks but the effective one does not. Traditional FL has no such accumulation pathway beyond the fixed model itself, and its constant benign majority steadily dilutes the planted trigger, which is why its ASR decays. Second, because the slice each client trains changes every round, two clients almost never optimize the same coordinates at the same time, so the per-round signal that a defender would use to tell a compromised update from an honest one is buried under subnetwork-topology variance. The next section shows that this second property is precisely why the standard FL defenses, applied unmodified or adapted to SuperFL, recover almost none of the lost ground.

3. Threat Model

We frame our threat model by defining the adversary’s goals and capabilities within the context of SFL.

Adversary Goal. The adversary’s goal is to compromise the central server’s global model, through poisoning attacks on adversarially-controlled clients. Throughout this paper, we equate the adversary’s goal in the traditional-federated setting, to poisoning of the MaxNet in the super-federated learning setting, where MaxNet is defined as the largest subnetwork $s_{max} \in S$. This paper solely considers backdoor attacks, where the adversary manipulates controlled-client models’ parameters to produce attacker-chosen outputs for specific inputs containing a trigger in the global model. For a given input x , the benign global MaxNet G_{max} , and the compromised MaxNet G'_{max} , the goal is:

$$f(G'_{max}, x) = \begin{cases} c_{target} & \text{if } x \text{ contains the trigger,} \\ f(G_{max}, x) & \text{otherwise.} \end{cases}$$

where, c_{target} is the malicious target label chosen by the adversary. While trigger-containing inputs should output c_{target} , the adversary’s goal is to also maintain high benign accuracy for the model (on non-trigger-containing inputs). Importantly, the adversary is **not** attempting to backdoor

specific subnetworks, but aims to target only the global model. This is distinct from stealthy attacks against SuperNets [25] which aim to target a specific subnetwork for attack while remaining hidden when other subnetworks are activated in deployment. We consider such attacks out of the scope of this work.

Adversarial Capabilities and Assumptions. First, we assume that the adversary is unaware that the central server is building a global model using a super-federated training approach, as prior attack and defense works for federated learning [2]–[14], [17]–[21], [31] strictly assume a traditional-federated learning setting. We discuss the implications of an adversary being aware that the global model is being trained in a super-federated setting in §7. Second, we assume that the adversary has full control over a fraction of the participating clients, k , in any given round where $k < \frac{N}{2}$. Notably this means that if the sample rate of clients s in a round is less than N , it is possible that for a single round this assumption breaks (e.g., if eight clients are controlled by the adversary, there are a total of 20 clients, and eight clients are sampled per round, it is possible that 4-8 adversarially-controlled clients are sampled in a given training round)[DS: why 4-8? could be 0 as well right?]. The adversary’s controlled clients does not change across training rounds. For controlled clients, the adversary manipulates the clients’ local data, training processes, and model updates before they are sent to the central aggregator. Third, the adversary has no control over the central aggregation server or non-controlled benign clients and cannot influence the aggregation process directly. Finally, we assume the adversary can employ an adaptive attack strategy. Knowing that FL-defenses are commonly used, the adversary may alter their attack strategy to circumvent it, for instance, by trying to make their malicious updates appear more similar to benign ones (from adversary-controlled clients training exclusively on benign data) to avoid being flagged as anomalous.

4. Classical Defenses Do Not Transfer

Federated learning already has a mature library of poisoning defenses, so the natural response to §2.2.2 is to reach for one. We show that this does not work. The three canonical, structurally distinct FL defenses recover almost none of the ground SuperFL loses, and they fail even after we rebuild each one to remove the specific artifact that breaks it. This is the second half of our motivating study, and it is why a SuperFL defense has to be built rather than borrowed.

The Defenses. We take three widely used defenses that span the design space of FL defense. FLAME [14] clusters client updates by pairwise cosine distance, clips the admitted updates to a median norm, and adds calibrated Gaussian noise. FoolsGold [31] lowers the aggregation weight of clients whose updates are mutually too similar, on the theory that sybils sharing an objective look alike. Auror [17] clusters the per-feature distribution of updates and discards the minority cluster on the features that most separate clients. Every one

TABLE 1: How We Adapted Each Classical Defense to SuperFL, and Why It Still Fails (CIFAR-10, 20 clients, 2 compromised, IID). Each adaptation removes the topology artifact that breaks the unmodified defense, yet the underlying detection still collapses once training matures, leaving the deployed MaxNet backdoored.

Defense	SuperFL adaptation	Why it still fails	ASR ↓	Acc. ↑
ModFLAME [14]	HDBSCAN distance matrix uses overlap-exclusive cosine; Gaussian noise is scaled per coordinate by its coverage; median clip and coverage-aware aggregation kept.	Cluster drift: once subnetwork-specific parameter distributions stabilize (round ~ 50), the compromised clients’ overlap distances merge into the benign majority and the dense-cluster test admits them.	0.81	0.86
ModFoolsGold [31]	Per-round overlap cosine accumulated as an EMA (avoiding the zero-padding artifact of full-vector history); aggregation denominator made coverage-aware.	Signal saturation: as training matures the per-round cosines compress into a narrow band, the logit reweighting saturates, and every client receives weight 1, i.e. plain FedAvg.	0.73	0.86
ModAuror [17]	Per-feature k -means restricted to the clients covering that feature; suspicion scored as a coverage-normalized rate rather than an absolute count.	Indicative-feature staleness: the features fingerprinted early stop discriminating as the model converges, and the suspicion ranking inverts after round ~ 150 .	0.78	0.86

MaxNet ASR and benign accuracy are the converged values for the SuperFL-aware port of each defense. The undefended SuperFL baseline is ASR 0.96 at benign accuracy 0.85, so the best adapted defense (ModFoolsGold) removes under a quarter of the attack.

of these compares or aggregates client updates as vectors over a single, fixed parameter space, the assumption that holds in traditional FL and breaks under subnetwork rotation, where the coordinates one client trains are mostly untouched by the next.

Unmodified Defenses Fail Silently or Catastrophically.

Run as published, the defenses either destroy the model or do nothing. FLAME’s noise calibration assumes a single coherent update per client, so on the rotating updates its injected-noise norm explodes from 2,143 to 123,629 within two rounds and then to NaN, collapsing benign accuracy to 0.10. Its resulting ASR of 0.0 is not a defense, because there is no model left to attack. FoolsGold and Auror fail in the opposite direction by flagging essentially no one: Auror identifies zero clients as malicious in 995 of 995 detection rounds and leaves MaxNet ASR at 0.76, a reduction of under 20 percentage points that comes from incidental aggregation effects rather than detection, and FoolsGold’s similarity test, computed across mismatched subnetwork supports, cannot separate sybils from honest clients at all. These outcomes are why a faithful comparison requires repairing each defense rather than reporting its unmodified failure.

Adapting Each Defense to SuperFL. We build a SuperFL-aware port of each defense, summarized in Table 1, that fixes the precise step where rotation interferes while leaving the rest of the method intact. For ModFLAME, we replace the full-vector cosine in the HDBSCAN distance matrix with the overlap-exclusive cosine of §5.1, so two clients are judged by their agreement on shared parameters rather than by which parameters they happened to share, and we scale the Gaussian noise at each coordinate by how many clients covered it, so that sparsely covered positions are not swamped by noise calibrated for the dense core. For ModFoolsGold, we replace the full-vector history accumulation, which reintroduces the zero-padding artifact once a client’s cumulative mask spans many subnetworks, with an exponential moving average of

the per-round overlap cosine, and we make the aggregation denominator coverage-aware so a suppressed client is suppressed only at the positions it actually touched. For ModAuror, we restrict the per-feature clustering to the set of clients that covered each feature, skipping features with too few contributors, and we score suspicion as the fraction of a client’s own covered features that look anomalous rather than an absolute count, which removes the bias against clients that drew small subnetworks. Each port runs without diverging and detects both compromised clients for the first tens of rounds.

Even Adapted, They Barely Dent the Attack. The repaired defenses do better than the unmodified ones but not nearly well enough, as the right two columns of Table 1 show. The strongest of the three, ModFoolsGold, only pulls MaxNet ASR down to 0.73; ModAuror reaches 0.78 and ModFLAME 0.81, all at a benign accuracy of about 0.86. Against an undefended baseline of 0.96, the best classical defense available, even rebuilt for the setting, removes barely a quarter of the attack, and the backdoor remains overwhelmingly successful. The failure mode differs by defense but the timing is shared: each one works for the first tens of rounds and then collapses as training matures. ModFLAME’s clusters drift once subnetwork distributions stabilize and the compromised clients merge into the benign majority; ModFoolsGold’s per-round cosines compress into a band so narrow that its reweighting saturates and reduces to plain FedAvg; and ModAuror’s early indicative features stop discriminating, inverting its suspicion ranking after round 150.

Why They Cannot Win Here. The common cause is that all three read a signal that subnetwork rotation steadily erases. Clustering and contribution-similarity assume two honest clients look alike and two sybils look alike, but in SuperFL two honest clients that trained disjoint subnetworks look dissimilar while two sybils that shared a large subnetwork look similar, so the axis the defenses sort on tracks which

subnetwork each client drew more than what it intended. Robust aggregation fares no better, because a coordinatewise median presumes every client contributes to every coordinate, whereas in SuperFL each coordinate is covered by only the few clients whose subnetwork included it, so the median is taken over too few honest votes to be robust. And noise calibration keyed to the largest benign update has no stable referent when update magnitude swings with subnetwork capacity from round to round. The defenses are not weak; they solve a problem, one fixed model with every client on every weight, that SuperFL no longer poses. The next section shows that the same rotation that breaks them opens an axis of defense they never had access to.

5. Design

§4 showed that classical defenses fail in SuperFL for one reason: comparing client updates across rotating subnetworks is dominated by which subnetwork each client happened to draw rather than by what each client tried to learn. Yet §2.2.2 also showed that the adversary’s poison must pass through parameters that other subnetworks share, since that is how it reaches the MaxNet. SFG is built on the observation that both facts point at the same object, the overlap between subnetworks. The overlap is where a meaningful comparison survives the rotation, and it is also the surface the adversary cannot avoid. We develop this into a defense in five steps. We first define a similarity that lives on the overlap and recovers the trust signal rotation destroys (§5.1). We then act on that signal by routing consistently low-trust clients onto the smallest shared subnetwork (§5.2), persist that decision once it is made (§5.3), and reduce a routed client’s weight in proportion to its anomaly (§5.4). Because all four of these steps depend on the trust signal being readable, we finally add a clip on the shared core that bounds the poison even when the signal is gone (§5.5). Throughout, we follow a running example of 20 clients of which two, denoted c_8 and c_{17} , are compromised. Figure 3 contrasts the two settings that make this possible: where traditional FL can only reweight client updates after a round (①), SuperFL lets the server also choose which subnetwork each client trains next (②), so it can route a suspected client to a small, contained subnetwork (③) while letting trusted clients train larger ones (④).

5.1. Overlap-Exclusive Cosine Similarity

To compare two clients without the comparison being swamped by their subnetwork shapes, we restrict it to the parameters both clients actually trained. Let the global SuperNet have dimension d , and let client i at round t train the coordinate set $S_i^{(t)} \subseteq \{1, \dots, d\}$, where $S_i^{(t)} = \{1, \dots, d\}$ recovers traditional FL. Write $\Delta_i^{(t)}$ for i ’s update over $S_i^{(t)}$, and let $\Delta_i^{(t)}|_A$ denote that update restricted to a coordinate set A . For two clients i and j the overlap is $O_{ij}^{(t)} = S_i^{(t)} \cap S_j^{(t)}$, and the overlap-exclusive

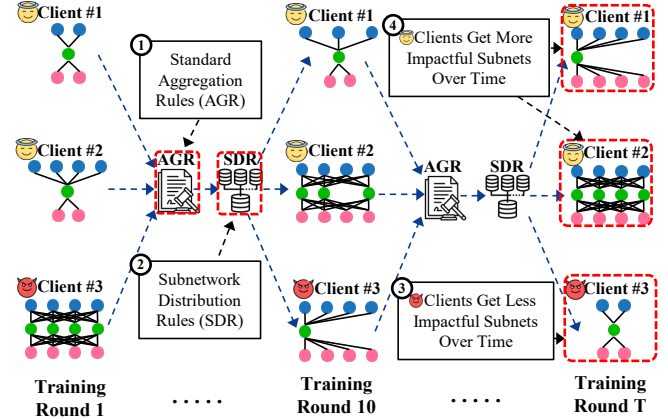


Figure 3: The New Axis of Defense. In traditional FL the server’s only lever after a round is how it aggregates updates (①). SuperFL adds a second lever: the server also chooses which subnetwork each client trains next (②), so it can route a suspected client to a small, contained subnetwork (③) while letting trusted clients train larger ones (④).

cosine similarity for the round is the cosine taken on the shared support alone:

$$c_{ij}^{(t)} = \cos\left(\Delta_i^{(t)}|_{O_{ij}^{(t)}}, \Delta_j^{(t)}|_{O_{ij}^{(t)}}\right). \quad (1)$$

Computing the cosine on $O_{ij}^{(t)}$ means the score reflects agreement on coordinates both clients optimized rather than agreement on coordinates one of them never touched. As a check, for $u = (\perp, 1, 2, 4)$ and $v = (4, 1, 2, \perp)$, where $\perp$ marks an untrained coordinate, the overlap is the middle two coordinates and Equation 1 gives $\cos((1, 2), (1, 2)) = 1.0$, the correct verdict that the two agree where they overlap, whereas zero-padding the absent coordinates would dilute the score toward 0.

A high cosine taken on a tiny overlap, or on near-zero updates, carries little information, so we weight each round’s score by the size of the overlap and the magnitudes of the two updates on it:

$$w_{ij}^{(t)} = |O_{ij}^{(t)}| \cdot \|\Delta_i^{(t)}|_{O_{ij}^{(t)}}\| \cdot \|\Delta_j^{(t)}|_{O_{ij}^{(t)}}\| \cdot |c_{ij}^{(t)}|. \quad (2)$$

The first factor upweights rounds where the clients shared more of the model, the norms upweight rounds with substantive updates, and $|c_{ij}^{(t)}|$ upweights rounds where the directional signal is strong in either alignment or opposition relative to rounds where the updates are nearly orthogonal. Over the window $\mathcal{T}$ of the most recent evaluation interval, we reduce the per-round scores to a single weighted similarity:

$$\tilde{C}_{ij} = \frac{\sum_{t \in \mathcal{T}} w_{ij}^{(t)} c_{ij}^{(t)}}{\sum_{t \in \mathcal{T}} w_{ij}^{(t)}}. \quad (3)$$

This quantity is symmetric, bounded in $[-1, 1]$, and invariant to the particular subnetwork shapes drawn in any one round, since each $c_{ij}^{(t)}$ is computed on whatever support the two clients shared that round. Placing $|c_{ij}^{(t)}|$ inside the weight

rather than $c_{ij}^{(t)}$ itself is deliberate: a round in which two clients pull hard in opposite directions contributes a large negative term to the numerator and a large positive term to the denominator, so consistent disagreement registers as a low or negative $\tilde{C}_{ij}$ instead of washing out. When every client trains the full model, the overlap factor becomes constant and $\tilde{C}_{ij}$ reduces to a magnitude-weighted moving average of ordinary cosine similarities, the traditional-FL quantity, so the measure degrades gracefully to the familiar one when there is no rotation to correct for.

Reducing the pairwise scores to a per-client trust $\bar{s}_i = \frac{1}{N-1} \sum_{j \neq i} \tilde{C}_{ij}$ exposes the compromised clients. Around round 10 the benign mean is 0.74 while c_8 reads 0.66 and c_{17} reads 0.67, roughly three and two and a half standard deviations below their peers. The separation is genuine, but it is also small and short-lived, which shapes how the defender can use it in the next three subsections.

5.2. Dissimilarity-Ranked Quarantine to the Core

A trust score is only useful if the server can act on it, and the obvious action, sorting clients into “benign” and “malicious” and discarding the latter, is exactly the action that ruined the classical defenses. A hard split pays catastrophically for every mistake, because the trust gap is narrow (§5.1) so mistakes are frequent, and a misjudged honest client loses its entire contribution. SFG therefore never makes a binary call. It ranks all 20 clients by trust $\bar{s}_i$ in ascending order and treats rank, not a label, as the signal, so a client that is mis-ordered by one position is handled almost identically to its neighbor.

The lowest-trust clients are routed to the *MinNet*, the smallest subnetwork in the pool. By the nested construction of the SuperNet (§2), the MinNet is contained in every other subnetwork and is therefore the universal minimal overlap of the whole family; in our SuperNet it is 14.5% of the parameters. Routing a suspect there is the strongest lever SFG has, because it does not down-rank the suspect’s update after the round. It denies the suspect access to all but the shared core *before* it trains, so whatever poison the suspect produces is confined to the core and is never written into the MaxNet-exclusive weights at all. This directly closes the subnetwork-size pathway that §2.2.2 identified as the route the poison takes to the MaxNet.

How many clients to quarantine should not require knowing how many attackers are present, an assumption the classical detectors of §4 quietly relied on. Each round, SFG fits a two-way split to the trust values $\{\bar{s}_i\}$ and quarantines the low-trust group, so the cohort size is read from the data. A floor keeps a small cohort under quarantine when the split is degenerate, and a ceiling forbids quarantining a majority, consistent with the honest-majority assumption of §3. Because $\bar{s}_i$ is computed only from the recent window and a routing decision is never carried forward as evidence, a client sent to the core for one round is judged afresh the next, which prevents the structurally lower similarity of a MinNet client from reinforcing its own quarantine.

With this rule, the data-driven split routes the clearly anomalous c_8 to the core in nearly every round and the fainter c_{17} in roughly half, while the most-misrouted honest client is sent there about one round in ten. When the attacker count is instead supplied to the server, the routing tightens further, placing c_{17} in the large majority of rounds and holding honest clients under 3%. The remaining gap, that c_{17} is routed only intermittently once its signal weakens, motivates the next subsection.

5.3. Sticky Suspicion

Routing depends on the trust signal, and §5.1 already warned that the signal is loud only while the backdoor is being injected. Once the trigger is planted, a compromised client has little left to push, so its updates relax toward the benign direction and the similarity that made it stand out decays to the benign baseline within roughly a hundred rounds. A server that re-decides from scratch each round therefore releases an attacker at precisely the moment when releasing it lets the backdoor resume. There is also an asymmetry that works for the adversary: an attacker that slips quarantine for even a few rounds contributes a full-strength poison gradient, which nudges the global direction toward its own and erodes the very signal that would re-flag it.

SFG breaks this loop by giving suspicion a short memory. A client is quarantined this round if it has been flagged in at least K of the last W rounds, not only if it is flagged in the current round. This carries a decision made while the signal was strong through the long stretch where it has faded, so an attacker can no longer launder its way back to trust by lying low. The memory is short and revocable rather than permanent, so a client that genuinely stops looking suspicious for a sustained stretch ages out of the window and an honest client caught by a transient dip is not condemned forever. Larger W and K catch a harder-to-pin attacker at the cost of holding the occasional honest client longer. In our running example, restoring c_{17} ’s memory recovers most of the quarantine coverage that the decaying signal would otherwise surrender, while the worst honest client’s quarantine rate stays low.

5.4. Rank-Proportional Down-Weighting

Quarantine bounds where a suspect’s update can land, but a quarantined client still contributes to the core during aggregation, and the core is the leak path of §2.2.2. Zeroing every quarantined client’s contribution would reintroduce the binary-classification mistake at the aggregation stage, silencing an honest client caught by the window. SFG instead scales a quarantined client’s aggregation weight by how far its trust sits below the benign consensus. Letting z_i be the standardized distance of $\bar{s}_i$ below the per-round mean trust, a quarantined client receives weight

$$\beta_i = \max(1 - k z_i, \beta_{\min}), \quad (4)$$

so a client far below the consensus is damped toward a floor $\beta_{\min}$ while one only slightly below keeps most of its weight, and clients outside quarantine keep full weight. The floor guarantees that no client is ever fully erased, preserving the graceful degradation that separates SFG from the classifiers of §4. This lever is intentionally secondary: it sharpens the quarantine for the most blatant attacker but, like the trust signal it derives from, weakens as that signal fades, which is why the structural quarantine of §5.2 rather than the weight carries the defense. In our running example the clearly anomalous c_8 is held near $\beta_{\min}$ for most of training, while c_{17} , whose trust drifts in and out of the benign band, is damped only intermittently.

5.5. Adaptive Core Clipping

Every mechanism so far rests on the trust signal being readable. §5.3 showed it fades after injection, and when many clients are compromised it disappears for a second reason: the attackers’ own mass shifts the consensus until no client looks anomalous at all. In exactly the regimes that matter most, a purely detection-driven defense has nothing left. SFG therefore adds a backstop that does not look at who is suspicious, and so holds regardless of the data distribution or the number of attackers.

The backstop clips contributions to the shared core to a robust magnitude the attackers cannot inflate. The core is the one region of the model with full participation, since every subnetwork contains it and every client therefore writes to it every round, which is precisely the population a robust statistic needs. SFG takes the median of the clients’ core-restricted update norms and rescales any core slice that exceeds it:

$$S_t = \text{median}_i \|\Delta_i^{(t)}|_{\text{core}}\|, \quad \Delta_i^{(t)}|_{\text{core}} \leftarrow \Delta_i^{(t)}|_{\text{core}} \cdot \min\left(1, \frac{S_t}{\|\Delta_i^{(t)}|_{\text{core}}\|}\right). \quad (5)$$

Because S_t is a median over all clients, an attacker minority cannot raise it, and a scaled-up poison contribution is pulled back to the benign magnitude. Because the clip reads only update norms, it is blind to the data distribution and needs no estimate of how many clients are compromised. This is the distribution- and count-agnostic guarantee that the detection layers lack, and it composes with them: routing decides where a suspect’s update lands, the clip bounds how large any core contribution can be, and the two together cap the leak through the universally shared parameters.

SFG clips rather than adding the calibrated noise a differential-privacy backstop would, because on a SuperNet core the clip is the active ingredient. Clipping bounds the leak at essentially no benign-accuracy cost, whereas noise calibrated to the clip bound is at best neutral and, once it is large enough to perturb the core, drives the median-norm calibration into a runaway that destroys the model. We quantify both effects in §6.4. The clip applies only to the small shared core and never to the larger subnetwork-exclusive weights, where each parameter is trained by too few clients per round for a median to be

meaningful; that restriction is itself a limitation we return to in §7.

5.6. SuperFedGuard in Practice

The five mechanisms compose into a single federated round, given in Algorithm 1. At the start of a round the server computes the overlap-exclusive similarity between every pair of clients over the recent window (Line 1) and reduces it to a per-client trust score (Line 2). It fits a two-way split to those scores and takes the low-trust group as the round’s quarantine cohort (Line 3), then folds in any client that has been flagged in at least K of the last W rounds so that a faded signal does not release a known suspect (Line 4). For each quarantined client, the server assigns the MinNet core for the next round (Line 6) and sets a down-weight proportional to the client’s anomaly (Line 7). When the clip backstop is enabled, the server computes the median core-norm over all clients (Line 10) and rescales every client’s core slice to it (Line 11). It then aggregates the updates under the down-weights (Line 13) and assigns subnetworks for the next round, granting trusted clients larger subnetworks while suspects stay on the core (Line 14). The only persistent state between rounds is the flag history that drives Line 4; everything else is recomputed from the current window, which is what keeps a single misjudgment from compounding. SFG requires only that the server can choose which model each client trains, so the same round applies to any weight-shared system with nested subnetworks, not only the SuperNets we evaluate (§7).

6. Evaluation

We evaluate SFG against the backdoor that defeated classical defenses in §4, and ask three questions. Does SFG recover the ground SuperFL losses to poisoning (§6.2)? How does that protection scale as more clients are compromised (§6.3)? And which of its mechanisms actually carry the defense (§6.4)? The headline answer is that SFG cuts the deployed model’s attack success rate from 0.96 to 0.30 on CIFAR-10 and from 1.00 to 0.30 on MNIST, a 69% and 70% relative reduction, at no cost to benign accuracy and without being told how many clients are compromised, where on CIFAR-10 the best classical defense managed only 24%.

6.1. Experimental Setup

SFG is implemented on top of the SuperFedNAS [1] SuperFL framework, whose SuperNet is an OFA ResNet-50 [22], [27] for 32×32 inputs with $\approx 71.8M$ parameters; its MinNet core is 14.5% of those parameters and, by the nested construction of the OFA search space, is contained in every subnetwork. Each round the server samples subnetworks from a pool spanning four capacity tiers, from the MinNet (per-block depth 0, expansion 0.10) to the MaxNet (depth 2, expansion 0.25). We train with 20 clients, all participating each round, on CIFAR-10 [29] and

Algorithm 1: SuperFedGuard: One Federated Round

Input: Global SuperNet θ ; client updates $\{\Delta_i\}_{i=1}^N$ at round t ; trust window $\mathcal{T}$; flag history H ; sticky parameters (W, K) ; MinNet $s_{\min}$; clip enabled
Output: Aggregated SuperNet θ' ; next-round subnetwork assignment A_{t+1}

```

// Overlap-exclusive trust over the recent window
1  $\tilde{C}_{ij} \leftarrow \text{OECS}(\Delta_i, \Delta_j, \mathcal{T}) \quad \forall i, j$ ;
// Reduce to a per-client trust score
2  $\bar{s}_i \leftarrow \frac{1}{N-1} \sum_{j \neq i} \tilde{C}_{ij}$ ;
// Data-driven low-trust cohort via a two-way split
3  $\mathcal{Q}_t \leftarrow \text{LowCluster}(\{\bar{s}_i\})$ ;
// Persist suspicion raised within the last  $W$  rounds
4  $\mathcal{Q}_t \leftarrow \mathcal{Q}_t \cup \{i : \sum_{r=t-W+1}^t H_{i,r} \geq K\}$ ;
5 for  $i \in \mathcal{Q}_t$  do
    // Quarantine the suspect to the shared core
    6  $A_{t+1}[i] \leftarrow s_{\min}$ ;
    // Down-weight by how far below the consensus it sits
    7  $\beta_i \leftarrow \max(1 - k z_i, \beta_{\min})$ ;
8 end
9 if clip enabled then
    // Median core-norm over all clients
    10  $S_t \leftarrow \text{median}_i \|\Delta_i|_{\text{core}}\|$ ;
    // Clip every client's core slice to  $S_t$ 
    11  $\Delta_i|_{\text{core}} \leftarrow \Delta_i|_{\text{core}} \cdot \min(1, S_t / \|\Delta_i|_{\text{core}}\|) \quad \forall i$ ;
12 end
// Weighted aggregation into the shared SuperNet
13  $\theta' \leftarrow \text{Aggregate}(\{\Delta_i\}, \{\beta_i\})$ ;
// Trusted clients receive larger subnetworks next round
14  $A_{t+1} \leftarrow \text{Assign}(\theta', \mathcal{Q}_t)$ ;

```

MNIST [32] under IID partitioning, for 500 to 1000 rounds. The attack is a BadNets [30] backdoor: each compromised client embeds a green-square trigger into 10% of its images and flips their labels to an attacker-chosen target, the friendliest setting for a defender since the poison is present and consistent from the first round. We report two metrics: *MaxNet ASR*, the fraction of a fully triggered test set the deployed MaxNet classifies as the target, and *benign accuracy*, clean-input accuracy on the MaxNet. Our baselines are the undefended SuperFL system of §2.2.2 and the three SuperFL-aware classical defenses of §4. SFG adds only a pairwise-similarity computation and a median-norm clip to each round and trains no extra model, so its overhead over undefended SuperFL is negligible.

6.2. SFG Neutralizes the Backdoor

Table 2 reports the end-to-end comparison at the canonical 10% poisoned-client ratio. On CIFAR-10 the undefended MaxNet is almost fully backdoored at 0.96 ASR, and the three classical defenses leave it at 0.81, 0.78, and 0.73, reductions of 16%, 19%, and 24% that never bring the attack below a coin flip. SFG instead drives

TABLE 2: End-to-End Defense Comparison Against the Backdoor in SuperFL (20 clients, 2 compromised, IID, BadNets green-square trigger). SFG reduces the deployed MaxNet’s attack success rate far more than any classical defense adapted to the setting, at no benign-accuracy cost, and without being told how many clients are compromised.

Dataset	Defense	MaxNet ASR ↓	Benign Acc. ↑	ASR Reduc.
CIFAR-10	Undefended SuperFL	0.96	0.85	–
	ModFLAME [14]	0.81	0.86	16%
	ModAuror [17]	0.78	0.86	19%
	ModFoolsGold [31]	0.73	0.86	24%
	SFG	0.30	0.86	69%
MNIST	Undefended SuperFL	1.00	0.99	–
	SFG	0.30	0.99	70%

MaxNet ASR is the fraction of triggered test inputs classified as the attacker’s target on the deployed (largest) subnetwork; benign accuracy is clean-input accuracy. ASR reduction is relative to the undefended SuperFL baseline. “Mod” defenses are the SuperFL-aware ports of §4; no MNIST classical-defense run was conducted. SFG figures are at convergence and infer the compromised-client count from the data.

MaxNet ASR to 0.30, a 69% relative reduction and nearly three times the best classical defense, while benign accuracy is 0.86, statistically indistinguishable from the undefended 0.85. The pattern is sharper on MNIST, where the undefended backdoor is perfect (1.00 ASR): SFG cuts it to 0.30, a 70% reduction, with benign accuracy unchanged at 0.99. Crucially, SFG achieves this without the prior knowledge classical detectors assume: it reads the number of clients to quarantine from the data each round rather than being given the attacker count.

The reduction comes from quarantine fidelity, not from broad suspicion of honest clients. Tracing the two compromised clients in our running example, SFG routes c_8 to the core in 96% of rounds and c_{17} in the large majority, while the most-misrouted honest client is quarantined in under 3% of rounds when the attacker count is known and at most 11% when it is inferred. False positives are therefore rare and, because a quarantined client is confined to the core and down-weighted rather than dropped (§5.4), never silenced, the 0.86 benign accuracy is the direct consequence. The residual 0.30 ASR is the false-negative cost: the trust signal is loud only during backdoor injection (§5.3), so a sliver of poison is planted before quarantine engages and is never fully overwritten, an effect the sticky window mitigates but does not erase.

6.3. Scaling with the Poisoned-Client Ratio

Table 3 and Figure 4 sweep the number of compromised clients from 1 to 9 of 20 with the routing layers alone. At low ratios the defense is decisive: with one compromised client MaxNet ASR is 0.16 on CIFAR-10 and 0.10 on MNIST, and at the canonical two it is 0.53 and 0.30. These sweep values are read at round 500 as a fixed cross- n comparison point; by convergence the CIFAR run at two compromised clients settles to the 0.30 of Table 2. As the ratio climbs the protection erodes monotonically: CIFAR-10

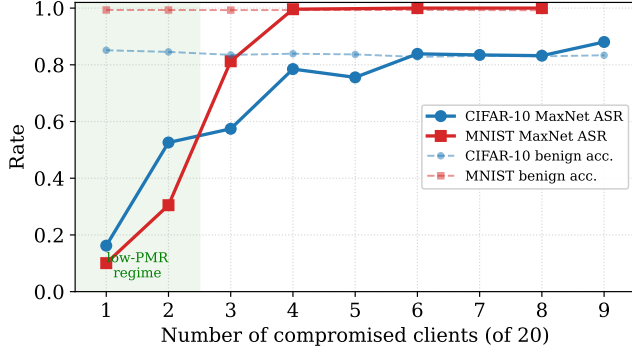


Figure 4: SFG Versus the Number of Compromised Clients. With routing only, the defense is strongest at low poisoned-client ratios and weakens as the ratio approaches the honest-majority bound; benign accuracy stays flat throughout.

TABLE 3: SFG Across the Poisoned-Client Ratio (20 clients, IID, routing only, MaxNet ASR and benign accuracy at round 500). The defense is strongest at low ratios and degrades as compromised clients approach the honest-majority bound, motivating the distribution-agnostic clip backstop of §5.5 (see Table 5).

Mal. (#)	Ratio (%)	CIFAR-10		MNIST	
		ASR ↓	Acc. ↑	ASR ↓	Acc. ↑
1	5%	0.16	0.85	0.10	0.99
2	10%	0.53	0.85	0.30	0.99
3	15%	0.57	0.84	0.81	0.99
4	20%	0.79	0.84	1.00	0.99
5	25%	0.76	0.84	—	—
6	30%	0.84	0.83	1.00	0.99
7	35%	0.83	0.83	—	—
8	40%	0.83	0.83	1.00	0.99
9	45%	0.88	0.83	—	—

“Routing only” = overlap-exclusive similarity, sticky quarantine, and rank-proportional down-weighting, with the core clip disabled. MNIST runs were conducted for $n \in \{1, 2, 3, 4, 6, 8\}$; “—” marks a configuration not run.

ASR rises through 0.79 at four clients to 0.88 at nine, and MNIST, whose easy-to-learn core makes its backdoor sharper, saturates at 1.00 by four compromised clients. Two facts make this an honest scaling rather than a collapse. First, benign accuracy never moves: it holds at 0.83–0.85 on CIFAR-10 and 0.99 on MNIST across the entire sweep, so the defense never trades task performance for robustness. Second, the erosion has a clear cause. As the compromised fraction grows toward the honest-majority bound, the attackers’ own mass shifts the trust consensus until no client looks anomalous, exactly the regime the detection-free backstop of the next subsection is designed for.

6.4. What Carries the Defense

Table 4 isolates the routing design. The lesson is that *how* suspects are chosen matters more than the rest of the pipeline. Ranking clients by dissimilarity and quarantining the lowest to the core, which is SFG’s choice, reaches 0.20

TABLE 4: Routing-Design Ablation (CIFAR-10, 20 clients, 2 compromised, IID, final MaxNet ASR). Each variant is given the true attacker count to isolate the routing strategy; the full system with an inferred count reaches 0.30 (Table 2). Continuous dissimilarity ranking with quarantine to the core is what works; binary classification of clients is worse than no defense because a single misclassification both silences an honest client and traps it in the core.

Routing strategy	Decision	ASR ↓	Acc. ↑
Un defended SuperFL	—	0.96	0.85
Rank → smallest drawn	continuous	0.71	0.84
Cluster → smallest drawn	binary	0.85	0.85
Cluster → core (forced)	binary	0.98	0.83
Rank → core (SFG)	continuous	0.20	0.87

TABLE 5: Core-Clip Backstop Ablation at the Honest-Majority Edge (CIFAR-10, 20 clients, 9 compromised, 45% ratio, IID, round 500). When the trust signal has collapsed, adaptive median clipping of the shared core is the active ingredient; added DP-style noise is at best neutral and diverges once it is large enough to perturb the core.

Configuration	MaxNet ASR ↓	Core ASR	Acc. ↑
Routing only	0.88	0.96	0.83
+ core clip	0.65	0.98	0.83
+ noise 0.5×	0.70	0.98	0.83
+ noise 10×	0.96	0.93	0.85
+ noise $\geq 25\times$	diverges (model destroyed)		

Core ASR is the attack success rate on the MinNet core itself. The noise multiple is the per-coordinate noise-to-signal ratio added on top of the clip.

final ASR, but replacing the continuous ranking with a binary cluster-and-forced-quarantine inverts the defense to 0.98, worse than the undefended 0.96, because a single misclassified honest client is both silenced and trapped in the core, where its lowered similarity re-flags it every round. Even a gentler binary variant that sends the suspect to the smallest *drawn* subnetwork only reaches 0.85. Continuous ranking is what avoids the cliff: a mis-ordering by one position is nearly harmless, and SFG’s 0.20 at 0.87 benign accuracy is the best cell in the table.

Table 5 turns to the hardest setting of nine compromised clients at a 45% ratio, where the trust signal has collapsed and routing alone leaves MaxNet ASR at 0.88. Adding the adaptive core clip pulls it to 0.65, a further 26% relative reduction, at no benign-accuracy cost (0.83) and, like the routing, with no estimate of the attacker count. The ablation also settles why we clip rather than noise: adding DP-style noise on top of the clip is at best neutral (0.70 at 0.5×) and becomes counterproductive as it grows (0.96 at 10×), because a per-round noise injection cannot erase a core that all nine attackers re-poison every round; once the noise is large enough to perturb the core it drives the median-norm calibration into a runaway that destroys the model entirely ($\geq 25\times$). Clipping is the active, distribution-agnostic ingredient; noise is not. Together, the routing of §6.2 and the clip here let SFG reduce the backdoor across both the low-ratio regime,

where detection suffices, and the high-ratio regime, where only the structural backstop holds.

7. Discussion and Limitations

Heterogeneous Client Data. We scoped our evaluation to IID client data (§3), and the protection SFG provides is weaker once that assumption is relaxed. Under non-IID data, benign clients with skewed local distributions naturally produce divergent updates, so the dissimilarity signal that drives quarantine can no longer cleanly separate an honest minority from compromised clients; we observe that the core clip, which removes 26% of the attack at a 45% poisoned-client ratio under IID data, recovers only a few percentage points under a Dirichlet split at $\alpha=0.1$. We hypothesize this is because heterogeneity pushes the leak out of the shared core and into the subnetwork-exclusive parameters that the core-restricted mechanisms cannot reach, while simultaneously creating benign update-clusters that the trust signal misreads. A defense for this regime likely needs a distribution-robust suspicion signal, density-based clustering that admits a benign majority rather than a fixed-size cohort, paired with a way to bound the exclusive parameters, which the next limitation describes.

The Coverage Gap. As observed in the backstop ablation (§6.4), the adaptive clip protects only the shared core, because a robust median norm requires a population of contributors and the core is the one region every client trains every round. The subnetwork-exclusive weights, those belonging to only the largest subnetworks, are trained by a single client per round, so they are statistical singletons with no population to clip against. A compromised client that escapes quarantine into a large subnetwork can therefore poison these thin parameters, and neither the clip nor the core mechanisms touch them. A promising fix is to alter the subnetwork distribution so that every exclusive parameter slice receives at least two contributors per round, for instance by distributing architectures in matched pairs, which would restore a minimal population for a robust statistic on exactly the weights the current backstop cannot defend. We leave the design of such coverage-complete distributions to future work.

Residual Poison and the Transient Signal. As observed in §6.2, SFG settles at 0.20–0.30 ASR rather than zero. The cause is that the trust signal is loud only while the backdoor is being injected and decays to the benign baseline within roughly a hundred rounds, so a thin layer of poison is planted before quarantine fully engages and is never completely overwritten; the effect is most pronounced on MNIST, whose easily learned core absorbs poison quickly. The sticky window already mitigates this by persisting an injection-phase decision, but it cannot recover a signal that was never strong. A detector tuned specifically to the injection burst, scoring the tight clique compromised clients form in early rounds, when their shared objective makes them maximally similar, could catch and persistently quarantine attackers before the first poison lands.

A SuperFL-Aware, Adaptive Adversary. Although not exercised in our evaluation, the adversary we study is deliberately the friendliest case: it is unaware that a SuperNet is being trained and uses a fixed trigger present from the first round, which maximizes the injection-phase signal SFG depends on. A SuperFL-aware adversary could decohere its compromised clients so they never form a detectable clique, or use a constrain-and-scale strategy that tunes updates to sit inside the benign distribution, defeating any detector. Here the value of the distribution-agnostic clip (§5.5) is that it bounds contribution magnitude regardless of whether detection succeeds, which is the same property that lets magnitude-based defenses blunt constrain-and-scale elsewhere; quantifying SFG against such an adaptive attacker is important future work.

Breadth of Evaluation. Our results cover one SuperNet family (OFA ResNet-50), two datasets (CIFAR-10 and MNIST), a single trigger pattern, and a 20-client deployment. These are sufficient to establish the mechanism, that subnetwork rotation can be turned from a liability into a defense, but a fuller validation would add a second SuperNet family such as an OFA MobileNetV3, a larger and partially participating client pool, additional trigger and attack types, and additional datasets such as GTSRB. We expect the structural mechanisms to transfer, since they depend on the nested-subnetwork property common to all SuperNets rather than on any dataset or architecture specifics, but confirming this is left to future work.

Generality. SFG’s only requirement of the underlying system is that a central server can adaptively choose which model each client trains in each round, and that those models share parameters with nested overlap. Any weight-shared collaborative-training system with these properties, not only the SuperNets we study, admits the same overlap-exclusive trust signal, the same quarantine-to-the-core routing, and the same shared-core clip, so the methodology is not specific to SuperFedNAS.

8. Related Work

Backdoor Attacks on Federated Learning. A large body of work plants backdoors in federated models by poisoning a minority of clients, from the model-replacement attack of Bagdasaryan et al. [2] to the distributed-trigger DBA of Xie et al. [9], alternating-minimization stealth [33], edge-case triggers [10], and the BadNets trigger [30] we adopt. These attacks assume the federated system trains a single shared architecture handed to every client each round; SFG studies how the same data-poisoning backdoor behaves when the system instead trains a weight-shared SuperNet, a setting these attacks were not designed for and in which, as §2.2.2 shows, they are even more effective.

Defenses for Federated Learning. Existing FL defenses fall into three families that SFG’s evaluation directly engages. Robust aggregation rules such as Krum [18] and coordinatewise median [19] bound a malicious minority by selecting or averaging toward a geometric center. Contribution-similarity defenses, including FoolsGold [31]

and the cosine-history methods of related work [11]–[13], score clients against one another. Feature-distribution and clustering auditors such as Auror [17] and FLAME [14] fingerprint each client’s update against a fixed reference. All three families compare or aggregate updates over a single parameter space that every client shares; §4 shows this assumption is what makes them ineffective under subnetwork rotation, and SFG differs in operating on the overlap two clients share and in adding the subnetwork-distribution control these methods do not have. SFG’s core clip is closest in spirit to FLAME’s median clipping, but where FLAME clips one coherent full-model update per client, SFG clips only the always-shared core, the one region of a SuperNet with the full participation a robust statistic needs.

Differentially Private Aggregation. Differential privacy has been used to mask malicious contributions by adding calibrated noise to the aggregate [20], [21], and FLAME couples such noise with clipping. Our ablation (§6.4) finds that on a SuperNet core the noise term is not the active ingredient and destabilizes training, so SFG retains the clip and omits the noise; the distinction matters because the core’s full participation lets a clip alone supply the magnitude bound that DP noise is usually added to guarantee.

Weight-Shared SuperNets. SuperNets were introduced for efficient neural architecture search and elastic deployment: once-for-all networks [22], CompOFA [23], and VNet [25] train a single weight-shared model containing a family of subnetworks, and SuperFedNAS [1] brings this to federated learning to lower training cost and raise accuracy. This line of work treats the subnetwork-distribution schedule purely as an efficiency mechanism; SFG repurposes that same schedule as a security primitive. The closest security work is VNet’s stealthy per-subnetwork backdoor [25], which hides an attack inside one chosen subnetwork while others behave normally, the inverse of our threat model, in which the adversary targets the deployed MaxNet and SFG defends it.

9. Conclusion

Super-Federated Learning was built to make federated training cheaper and more accurate, and we showed that it quietly makes the system easier to backdoor: the same subnetwork rotation that shrinks each round’s attack surface lets poison accumulate into the deployed model while hiding which client planted it, and it breaks the classical defenses that assume one fixed shared model. SFG turns that rotation back into a defense. By comparing client updates only on the parameters they jointly trained, quarantining persistently suspicious clients to the always-shared core, and clipping every contribution to that core to a robust magnitude, SFG reduces the deployed model’s attack success rate by up to 70% at no benign-accuracy cost and without knowing how many clients are compromised, where the strongest classical defense adapted to the setting removes barely a quarter of the attack. The server’s freedom to choose which model each client trains is a defensive control with no analogue in

traditional federated learning, and we believe it is a productive axis for securing any weight-shared collaborative system.

References

- [1] A. Khare, A. Agrawal, A. Annavajjala, P. Behnam, M. Lee, H. Latapie, and A. Tumanov, “Superfednas: Cost-efficient federated neural architecture search for on-device inference,” in *Proceedings of the 2024 European Conference on Computer Vision (ECCV)*, Malmö, Sweden, Sep. 2024.
- [2] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in *Proceedings of the 2018 International Conference on Artificial Intelligence and Statistics (AISTATS)*, Virtual Conference, Aug. 2020.
- [3] M. Fang, X. Cao, J. Jia, and N. Gong, “Local model poisoning attacks to byzantine-robust federated learning,” in *Proceedings of the 29th USENIX Security Symposium (Security)*, Virtual Conference, Aug. 2020.
- [4] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, “Data poisoning attacks against federated learning systems,” in *Proceedings of the 25th European Symposium on Research in Computer Security (ESORICS)*, Guildford, United Kingdom, Sep. 2020.
- [5] D. Cao, S. Chang, Z. Lin, G. Liu, and D. Sun, “Understanding distributed poisoning attack in federated learning,” in *Proceedings of the 2019 International Conference on Parallel and Distributed Systems (ICPADS)*, Tianjin, China, Dec. 2019.
- [6] X. Cao and N. Z. Gong, “Mpafl: Model poisoning attacks to federated learning based on fake clients,” in *Proceedings of the 39th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, New Orleans, Louisiana, Jun. 2022.
- [7] V. Shejwalkar, A. Houmansadr, P. Kairouz, and D. Ramage, “Back to the drawing board: A critical evaluation of poisoning attacks on production federated learning,” in *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*, San Francisco, CA, May 2022.
- [8] V. Shejwalkar and A. Houmansadr, “Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning,” in *Proceedings of the 2021 Annual Network and Distributed System Security Symposium (NDSS)*, Virtual Conference, Feb. 2021.
- [9] C. Xie, K. Huang, P.-Y. Chen, and B. Li, “Dba: Distributed backdoor attacks against federated learning,” in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, New Orleans, LA, May 2019.
- [10] H. Wang, K. Sreenivasan, S. Rajput, H. Vishwakarma, S. Agarwal, J.-y. Sohn, K. Lee, and D. Papailiopoulos, “Attack of the tails: Yes, you really can backdoor federated learning,” in *Proceedings of the 34th Conference on Neural*

- Information Processing Systems (NeurIPS)*, Virtual Conference, Dec. 2020.
- [11] A. E. Samy and Š. Girdzijauskas, “Mitigating sybil attacks in federated learning,” in *Proceedings of the 18th International Conference on Information Security Practice and Experience (ISPEC)*, Kongens Lyngby, Denmark, Aug. 2023.
 - [12] S. Awan, B. Luo, and F. Li, “Contra: Defending against poisoning attacks in federated learning,” in *Proceedings of the 26th European Symposium on Research in Computer Security (ESORICS)*, Virtual Conference, Sep. 2021.
 - [13] X. Li, Z. Qu, S. Zhao, B. Tang, Z. Lu, and Y. Liu, “Lomar: A local defense against poisoning attack on federated learning,” IEEE, 2021.
 - [14] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Mollering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, *et al.*, “Flame: Taming backdoors in federated learning,” in *Proceedings of the 31st USENIX Security Symposium (Security)*, Boston, MA, Aug. 2022.
 - [15] G. Xia, J. Chen, C. Yu, and J. Ma, “Poisoning attacks in federated learning: A survey,” *IEEE Access*, vol. 11, 2023.
 - [16] T. D. Nguyen, T. Nguyen, P. Le Nguyen, H. H. Pham, K. D. Doan, and K.-S. Wong, “Backdoor attacks and defenses in federated learning: Survey, challenges and future research directions,” *Engineering Applications of Artificial Intelligence*, vol. 127, 2024.
 - [17] S. Shen, S. Tople, and P. Saxena, “Auror: Defending against poisoning attacks in collaborative deep learning systems,” ser. ACSAC16.
 - [18] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, Dec. 2017.
 - [19] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Stockholm, Sweden, Jun. 2018.
 - [20] C. Dwork and A. Roth, “The algorithmic foundations of differential privacy,” *Foundations and Trends in Theoretical Computer Science*, vol. 9, pp. 211–407, 2014.
 - [21] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, “Learning differentially private recurrent language models,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, Vancouver, BC, Apr. 2018.
 - [22] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, “Once-for-all: Train one network and specialize it for efficient deployment,” in *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, Virtual Conference, Apr. 2020.
 - [23] M. Sahni, S. Varshini, A. Khare, and A. Tumanov, “Compofa: Compound once-for-all networks for faster multi-platform deployment,” in *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, Virtual Conference, May 2021.
 - [24] A. Khare, D. Garg, S. Kalra, S. Grandhi, I. Stoica, and A. Tumanov, *Superserve: Fine-grained inference serving for unpredictable workloads*, 2023. arXiv: 2312 . 16733 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2312.16733>.
 - [25] D. Oygenblik, A. Vemulapalli, A. Agrawal, D. Sanyal, A. Tumanov, and B. Saltaformaggio, “VillainNet: Targeted Poisoning Attacks Against SuperNets Along the Accuracy-Latency Pareto Frontier,” in *Proceedings of the 32nd ACM Conference on Computer and Communications Security (CCS)*, Taipei, Taiwan, Oct. 2025.
 - [26] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, *et al.*, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, Seoul, South Korea, 2019.
 - [27] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the 33rd IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, Nevada, Jun. 2016.
 - [28] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “The German Traffic Sign Recognition Benchmark: A multi-class classification competition,” in *IEEE International Joint Conference on Neural Networks*, 2011.
 - [29] *Cifar-10 (canadian institute for advanced research)*, <http://www.cs.toronto.edu/~kriz/cifar.html>, [Accessed: 2023-01-19].
 - [30] T. Gu, B. Dolan-Gavitt, and S. Garg, “Badnets: Identifying vulnerabilities in the machine learning model supply chain,” *arXiv preprint arXiv:1708.06733*, 2017. [Online]. Available: <https://arxiv.org/abs/1708.06733>.
 - [31] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “The limitations of federated learning in sybil settings,” in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, Virtual Conference, Oct. 2020.
 - [32] L. Deng, “The mnist database of handwritten digit images for machine learning research,” *IEEE Signal Processing Magazine*, vol. 29, 2012.
 - [33] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, “Analyzing federated learning through an adversarial lens.”